



SAE : Traiter des données

RABAH Soumaya et SONDAG Chloé

RT13

Besoin



Gestion de l'espace disque

Problème critique pour le fonctionnement du système



Conséquences

Installations impossibles, mises à mises à jour bloquées, pannes au pannes au démarrage



Solution

Identifier et supprimer les fichiers volumineux non utilisés





Difficulté de Localisation

Fichiers volumineux dispersés,
recherche manuelle risquée

Suppression Risquée

Danger de supprimer des fichiers
importants par erreur

Problématiques Majeures

Solution apportée par l'outil

1

Scan

Analyse de l'arborescence du répertoire

2

Filtre et Tri

Sélection et classement des fichiers volumineux

3

Illustre et Efface

Interface graphique pour gérer puis supprimer



Analyse des Fichiers (analyse_fichiers.py)

Objectif

Analyser et filtrer les fichiers par taille
taille

Code clé

Génération d'une liste des fichiers
fichiers volumineux

Sortie

Fichier JSON avec informations sur
fichiers volumineux

```
5
6 # Récupère, trie et filtre les fichiers du répertoire donné
7 def obtenir_fichiers(repertoire, taille_min, nb_max=100): 1 usage ± soumaya
8     taille_min = taille_min * 1024 * 1024 # Conversion Mo → octets
9     fichiers = [(str(f), f.stat().st_size) for f in pathlib.Path(repertoire).rglob('*') if f.is_file()]
10     return sorted([f for f in fichiers if f[1] > taille_min], key=lambda x: x[1], reverse=True)[:nb_max]
11
```

analyse_fichiers.py Creation_Boutons.py resultats.json main.py × Powershell.ps1

```

import subprocess
import json
import sys
import random
from PyQt5.QtWidgets import QApplication
from PyQt5.QtGui import QColor
from Creation_Onglets import Onglets
from Creation_Camembert import Camembert
from Creation_Legendes import Legendes
from Creation_Boutons import Boutons

NB_LEGENDES_PAR_PAGE = 25
FICHER_JSON = "resultats.json"

# Exécuter analyse_fichiers.py AVANT de continuer
subprocess.run(args=["python3", "analyse_fichiers.py"], check=True)

# Charge les données du fichier JSON et retourne une liste de tuples (nom, taille).
def charger_fichiers_json(nom_fichier): 1 usage ± soumaya
    with open(nom_fichier, "r") as f:
        return json.load(f)

# Génère une liste de couleurs aléatoires en format QColor.
def generer_couleurs(nb): 1 usage ± soumaya
    return [QColor(random.randint(a: 0, b: 255), random.randint(a: 0, b: 255), random.randint(a: 0, b: 255))]

# Génère un script PowerShell pour supprimer les fichiers sélectionnés.
def creation_script_suppression(): 1 usage ± soumaya
    fichiers_a_supprimer = []
    for legende in liste_legende:
        etats = legende.recupere_etats_cases_a_cocher()
        fichiers_a_supprimer.extend([f[0] for f, etat in zip(liste_fichiers, etats) if etat])

    if not fichiers_a_supprimer:
        print("Aucun fichier sélectionné pour la suppression.")
        return

```

Interface Utilisateur (main.py)



Récupération

Chargement du fichier JSON



Interface

Graphiques et interaction utilisateur
utilisateur



Fonctionnalités

Pagination, génération script PowerShell

Script PowerShell

1

Vérification

Installation de Python

2

Exécution

Lancement du script main.py

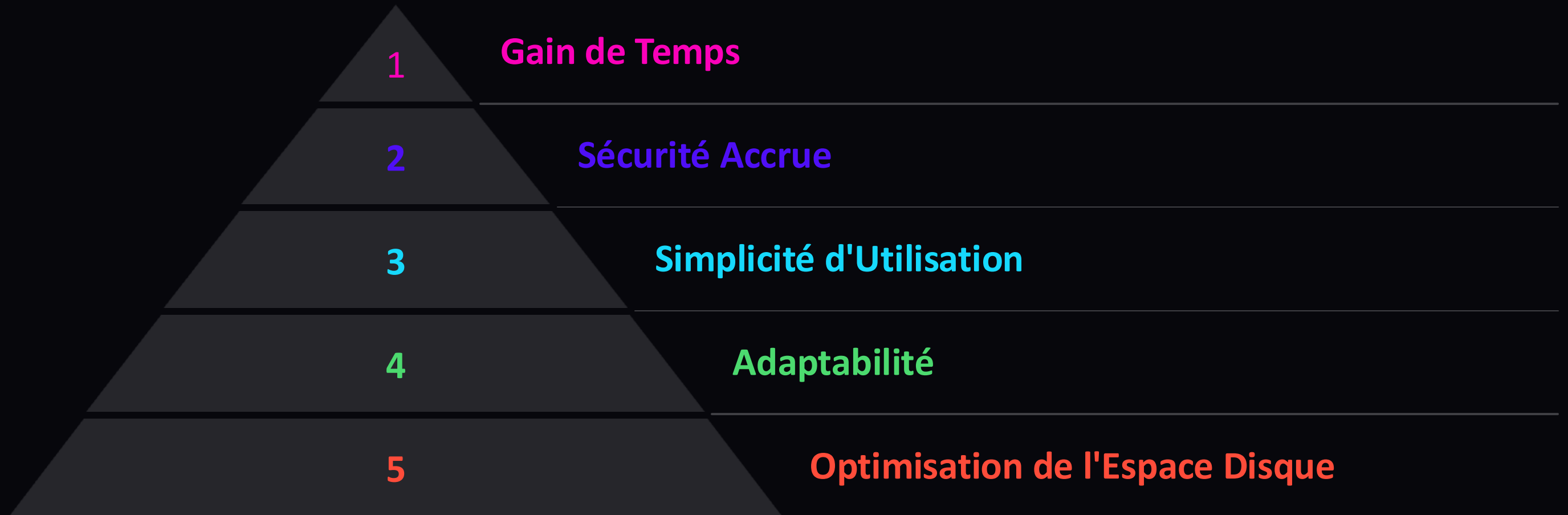
3

Sécurité

Confirmations avant suppression



Avantages



Difficultés rencontrés

Différences

Cmd Powershell



Comception

Sélection des fichiers



Compatibilité

macOS et Windows



Instalation

Des dépendances



Erreur

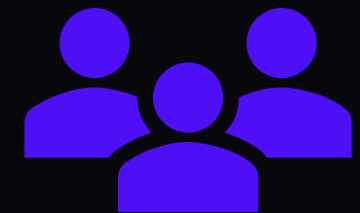
Valeur de filtrage hautes



Conclusion



Objectifs Atteints



Solution Pratique pour les
Administrateurs Systèmes